

Winventory

BDR-Projet



Lestiboudois Maxime &
Parisod Nathan &
Surbeck Léon
24.01.2025

Contents

Introduction	2
Modèle EA	3
Modèle relationnel	3
Création de tables	5
Remplissage des tables	8
Description de l'application	10
Accès à l'application	10
Les différentes page de l'application	10
Accès aux pages	10
Page de connexion	13
Page d'enregistrement	14
Page d'accueil	15
Vue des stocks	16
Formulaire de commandes	17
Formulaire de ventes	18
Fournisseurs	19
Ajouter un fournisseur	20
Quelques spécificités	21
Bugs connus	21
Conclusion	21
Annexes	23
.1 Annexe 1 - Accès à l'application, DAI	23

Introduction

Le projet **Winventory** a pour ambition de concevoir une base de données relationnelle performante et innovante, spécialement pensée pour gérer les inventaires de vins, bières, spiritueux, boissons non alcoolisées, et autres liquides destinés à la consommation.

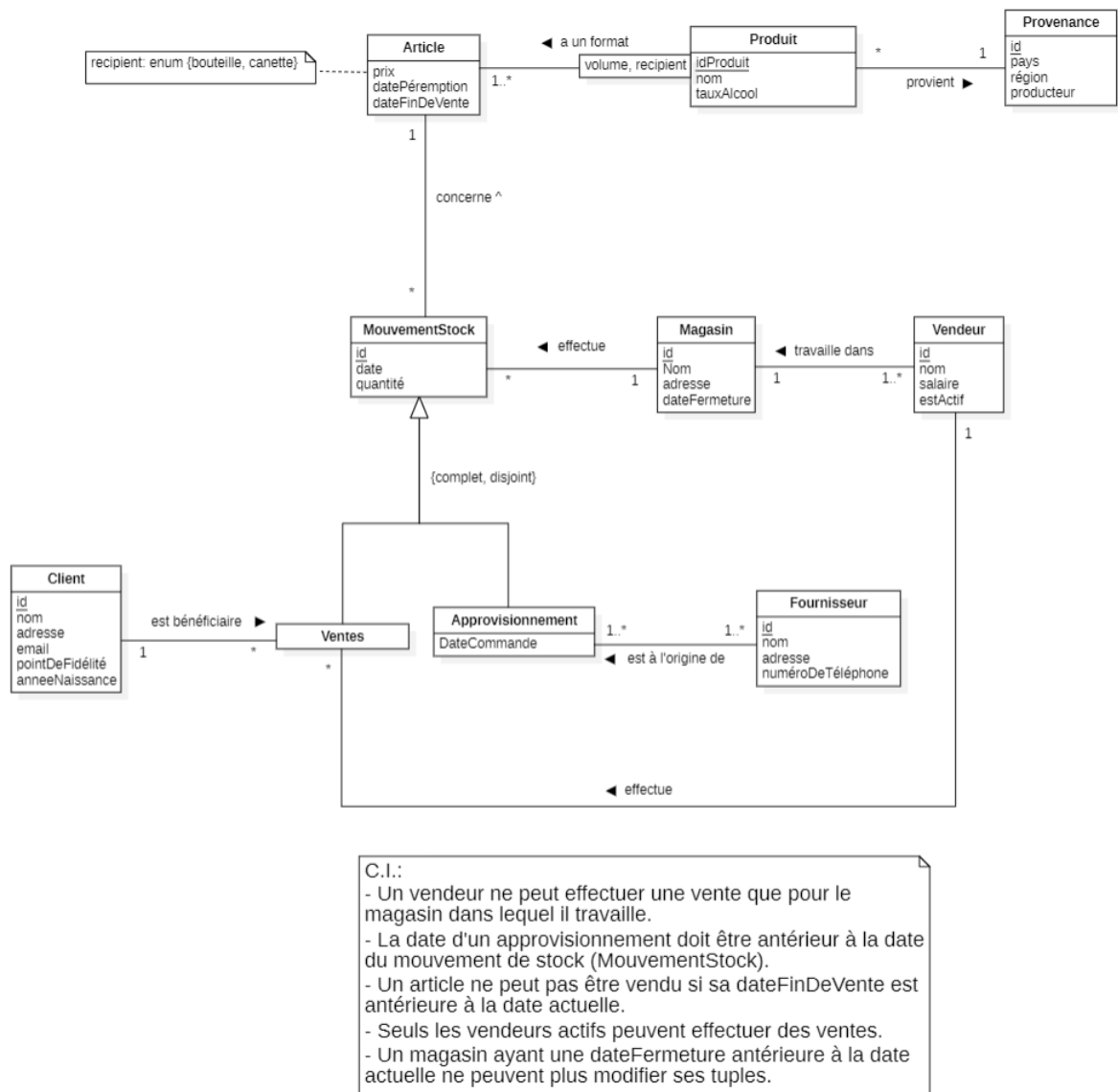
Grâce à une solution centralisée, **Winventory** simplifie la gestion des stocks, le suivi des acquisitions/commandes et les retraits/ventes de produits, dans le but d'offrir à terme des fonctionnalités avancées d'analyse et de reporting via une interface utilisateur intuitive.

Le système offre une gestion centralisée de l'inventaire d'un magasin. Les vues des stocks offertes par l'application permettent de mieux anticiper les besoins et d'optimiser la préparation des stocks.

Winventory s'appuie sur la robustesse de PostgreSQL pour garantir une performance optimale dans le traitement des requêtes.

Toutes les requêtes SQL sont écrites manuellement afin de garantir un contrôle total et d'assurer une transparence complète du système.

Modèle EA



Modèle relationnel

- **Provenance**

- id, pays, région, producteur

- **Produit**

- idProduit, idProvenance, nom, tauxAlcool

- *idProvenance* référence *Provenance.id*, *idProvenance* NOT NULL

- **Article**

- idProduit, volume, recipient, prix, datePeremption, dateFinDeVente
- *idProduit* référence *Produit.id*

- **MouvementStock**

- id, idMagasin, idProduit, volume, recipient, date, quantite
- *idMagasin* référence *Magasin.id*
- *idProduit*, *volume*, *recipient* référence *Article.idProduit*, *volume*, *recipient*
- *idProduit*, *volume*, *recipient* NOT NULL

- **Magasin**

- id, nom, adresse, dateFermeture

- **Vendeur**

- id, idMagasin, nom, salaire, estActif
- *idMagasin* référence *Magasin.id*, *idMagasin* NOT NULL

- **Vente**

- idMouvementStock, idVendeur, idClient
- *idMouvementStock* référence *MouvementStock.id*
- *idVendeur* référence *Vendeur.id*, *idVendeur* NOT NULL
- *idClient* référence *Client.id*, *idClient* NOT NULL

- **Approvisionnement**

- idMouvementStock, dateCommande
- *idMouvementStock* référence *MouvementStock.id*

- **Client**

- id, nom, adresse, email, pointDeFidelite, annéeNaissance
- *email* UNIQUE

- **Fournisseur**

- id, nom, adresse, numeroTelephone

- **Approvisionnement_Fournisseur**

- idMouvementStock, idFournisseur
- *idMouvementStock* référence *MouvementStock.id*
- *idFournisseur* référence *Fournisseur.id*

Création de tables

Voici le script SQL nous permettant de créer les tables de notre base de données:

```
CREATE TABLE IF NOT EXISTS Provenance(  
    id SERIAL,  
    pays VARCHAR(80) ,  
    region VARCHAR(80) ,  
    producteur VARCHAR(80) ,  
    CONSTRAINT PK_Provenance PRIMARY KEY (id)  
);  
  
—CREATE TYPE typeRecipient AS ENUM ( 'bouteille ', 'canette ' );  
  
CREATE TABLE IF NOT EXISTS Produit(  
    idProduit SERIAL,  
    idProvenance INTEGER NOT NULL,  
    nom VARCHAR(80) NOT NULL,  
    tauxAlcool DOUBLE PRECISION NOT NULL,  
    CONSTRAINT PK_Produit PRIMARY KEY (idProduit) ,  
    CONSTRAINT FK_idProvenance foreign KEY(idProvenance)  
        references Provenance(id) ON UPDATE CASCADE ON DELETE  
        RESTRICT  
);  
  
CREATE TABLE IF NOT EXISTS Article(  
    idProduit INTEGER,  
    volume INTEGER,  
    recipient typeRecipient ,  
    prix DOUBLE PRECISION NOT NULL,  
    datePeremption DATE NOT NULL,  
    dateFinDeVente DATE,  
    CONSTRAINT PK_Article PRIMARY KEY (idProduit , volume ,  
        recipient) ,  
    CONSTRAINT FK_Article_Produit FOREIGN KEY (idProduit)  
        REFERENCES Produit(idProduit) ON UPDATE CASCADE ON DELETE  
        RESTRICT  
);  
  
CREATE TABLE IF NOT EXISTS Magasin(  
    id SERIAL,  
    nom VARCHAR(80) NOT NULL,  
    adresse VARCHAR(350) NOT NULL,  
    dateFermeture DATE,  
    CONSTRAINT PK_Magasin PRIMARY KEY (id)
```

);

```
CREATE TABLE IF NOT EXISTS MouvementStock(  
    id SERIAL,  
    idMagasin INTEGER NOT NULL,  
    idProduit INTEGER NOT NULL,  
    volume INTEGER NOT NULL,  
    recipient typeRecipient NOT NULL,  
    date DATE,  
    quantite INTEGER,  
    CONSTRAINT PK_MouvementStock PRIMARY KEY (id),  
    CONSTRAINT FK_MouvementStock_Magasin FOREIGN KEY (idMagasin)  
        REFERENCES Magasin(id) ON UPDATE CASCADE ON DELETE  
        CASCADE,  
    CONSTRAINT FK_MouvementStock_Article FOREIGN KEY (idProduit ,  
        volume, recipient) REFERENCES Article(idProduit , volume ,  
        recipient) ON UPDATE CASCADE ON DELETE CASCADE  
);
```

```
CREATE TABLE IF NOT EXISTS Vendeur(  
    id SERIAL,  
    idMagasin INTEGER NOT NULL,  
    nom VARCHAR(80) ,  
    salaire DOUBLE PRECISION,  
    estActif BOOL NOT NULL,  
    CONSTRAINT PK_Vendeur PRIMARY KEY (id),  
    CONSTRAINT FK_Vendeur_Magasin FOREIGN KEY (idMagasin)  
        REFERENCES Magasin(id) ON UPDATE CASCADE ON DELETE  
        CASCADE  
);
```

```
CREATE TABLE IF NOT EXISTS Client(  
    id SERIAL,  
    nom VARCHAR(80) ,  
    adresse VARCHAR(150) ,  
    email VARCHAR(100) UNIQUE,  
    pointDeFidelite INTEGER,  
    anneeNaissance INTEGER,  
    CONSTRAINT PK_Client PRIMARY KEY (id)  
);
```

```
CREATE TABLE IF NOT EXISTS Vente(  
    idMouvementStock INTEGER,  
    idVendeur INTEGER NOT NULL,  
    idClient INTEGER NOT NULL,  
    CONSTRAINT PK_Vente PRIMARY KEY (idMouvementStock) ,
```

```

    CONSTRAINT FK_Vente_MouvementStock FOREIGN KEY (
        idMouvementStock) REFERENCES MouvementStock(id) ON UPDATE
        CASCADE ON DELETE CASCADE,
    CONSTRAINT FK_Vente_Vendeur FOREIGN KEY (idVendeur)
        REFERENCES Vendeur(id) ON UPDATE CASCADE ON DELETE
        RESTRICT,
    CONSTRAINT FK_Vente_Client FOREIGN KEY (idClient) REFERENCES
        Client(id) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS Approvisionnement(
    idMouvementStock INTEGER,
    dateCommande DATE NOT NULL,
    CONSTRAINT PK_Approvisionnement PRIMARY KEY (
        idMouvementStock),
    CONSTRAINT FK_Approvisionnement_Approvisionnement FOREIGN
        KEY (idMouvementStock) REFERENCES MouvementStock(id) ON
        UPDATE CASCADE ON DELETE cascade,
    CONSTRAINT check_DateCommande CHECK (dateCommande <=
        CURRENTDATE)
);

CREATE TABLE IF NOT EXISTS Fournisseur(
    id SERIAL,
    nom VARCHAR(80),
    adresse VARCHAR(150),
    numeroTelephone VARCHAR(30),
    CONSTRAINT PK_Fournisseur PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS Approvisionnement_Fournisseur(
    idMouvementStock INTEGER,
    idFournisseur INTEGER,
    CONSTRAINT PK_Approvisionnement_Fournisseur PRIMARY KEY (
        idMouvementStock, idFournisseur),
    CONSTRAINT FK_Approvisionnement_Fournisseur_idMouvementStock
        FOREIGN KEY (idMouvementStock) REFERENCES
        Approvisionnement(idMouvementStock) ON UPDATE CASCADE ON
        DELETE CASCADE,
    CONSTRAINT FK_Approvisionnement_Fournisseur_idFournisseur
        FOREIGN KEY (idFournisseur) REFERENCES Fournisseur(id) ON
        UPDATE CASCADE ON DELETE CASCADE
);

```


Remplissage des tables

Ce script SQL suivant nous a permis de remplir les tables de notre base de données:

```
-- Remplissage de la table Provenance
INSERT INTO Provenance (pays, region, producteur) VALUES
('France', 'Bordeaux', 'Château Margaux'),
('Italie', 'Toscane', 'Antinori'),
('Espagne', 'Ribera del Duero', 'Bodega Vega Sicilia'),
('USA', 'Napa Valley', 'Robert Mondavi'),
('France', 'Champagne', 'Moët & Chandon');

-- Remplissage de la table Produit
INSERT INTO Produit (nom, tauxAlcool, idProvenance) VALUES
('Vin Rouge Bordeaux', 13.5, 1),
('Vin Blanc Chardonnay', 12.0, 2),
('Champagne Brut', 12.5, 2),
('Whisky Single Malt', 40.0, 3),
('Bière Blonde', 5.0, 1);

-- Remplissage de la table Article
INSERT INTO Article (idProduit, volume, recipient, prix,
    datePeremption, dateFinDeVente) VALUES
(1, 750, 'bouteille', 45.99, '2025-12-31', NULL),
(1, 1500, 'bouteille', 89.99, '2025-12-31', NULL),
(2, 750, 'bouteille', 35.99, '2024-11-30', '2024-10-01'),
(3, 750, 'bouteille', 55.00, '2026-06-30', NULL),
(4, 700, 'bouteille', 120.00, '2030-01-01', NULL),
(5, 330, 'canette', 2.50, '2023-12-31', '2023-11-01');

-- Remplissage de la table Magasin
INSERT INTO Magasin (nom, adresse, dateFermeture) VALUES
('Cave de Paris', '12 rue de la Paix, Paris, France', NULL),
('Wine World', '45 High Street, London, UK', NULL),
('Enoteca Roma', 'Via Condotti, 25, Rome, Italy', '2025-01-01');

-- Remplissage de la table MouvementStock
INSERT INTO MouvementStock (idMagasin, idProduit, volume,
    recipient, date, quantite) VALUES
(1, 1, 750, 'bouteille', '2024-01-10', 100),
(1, 2, 750, 'bouteille', '2024-01-15', 50),
(2, 3, 750, 'bouteille', '2024-01-20', 30),
(2, 5, 330, 'canette', '2023-12-01', 500),
(3, 4, 700, 'bouteille', '2023-11-25', 20);
```

```

-- Remplissage de la table Vendeur
INSERT INTO Vendeur (idMagasin, nom, salaire, estActif) VALUES
(1, 'Alice Dupont', 2500.00, TRUE),
(2, 'John Smith', 2200.00, TRUE),
(3, 'Giulia Rossi', 2400.00, FALSE);

-- Remplissage de la table Client
INSERT INTO Client (nom, adresse, email, pointDeFidelite,
anneeNaissance) VALUES
('Paul Morel', '34 avenue des Champs, Paris, France', 'paul.
morel@example.com', 120, 1985),
('Anna Garcia', '23 Carrer Major, Barcelona, Spain', 'anna.
garcia@example.com', 80, 1990),
('James Taylor', '56 Broadway, New York, USA', 'james.
taylor@example.com', 200, 1980);

-- Remplissage de la table Vente
INSERT INTO Vente (idMouvementStock, idVendeur, idClient) VALUES
(1, 1, 1),
(2, 1, 2),
(3, 2, 3);

-- Remplissage de la table Approvisionnement
INSERT INTO Approvisionnement (idMouvementStock, dateCommande)
VALUES
(4, '2023-11-01'),
(5, '2023-10-15');

-- Remplissage de la table Fournisseur
INSERT INTO Fournisseur (nom, adresse, numeroTelephone) VALUES
('Distrib Vins France', '45 rue du Vin, Bordeaux, France', '+33
5 56 00 00 01'),
('Champagne Select', '12 avenue de la Champagne, Reims, France',
'+33 3 26 00 00 02'),
('Global Spirits', '99 Whiskey Lane, Dublin, Ireland', '+353 1
00 00 03');

-- Remplissage de la table Approvisionnement_Fournisseur
INSERT INTO Approvisionnement_Fournisseur (idMouvementStock,
idFournisseur) VALUES
(4, 1),
(5, 3);

```

Description de l'application

L'application permet de gérer et de voir le contenu des stocks d'un magasin (pour l'instant).

Accès à l'application

Pour accéder à l'application, il y a deux manières possibles:

1. en local, il suffit de cloner le répertoire Github au lieu suivant : <https://github.com/La-Kirby-Team/BDR-Project>.
Une fois le répertoire cloné en local, il suffit de suivre les instructions contenues dans le fichier README.md (voir Annexe 1)
2. l'application est également disponible sur le site winventory.duckdns.org.

Tout le monde peut avoir accès à la page du magasin, à condition d'avoir créé un compte pour se connecter.

Les différentes page de l'application

L'application comprend plusieurs pages :

1. La page de connexion
2. La page d'enregistrement
3. La page d'accueil
4. La vue des stocks
5. Le formulaire de commandes
6. Le formulaire de vente
7. La vue des fournisseurs
8. Le formulaire d'ajout de fournisseur
9. La page profil du vendeur

Accès aux pages

La page de connexion est la page par défaut lorsque l'utilisateur se connecte au site et qu'il n'est pas encore connecté avec son compte.

La page d'accueil est accessible lorsqu'on vient de se connecter à l'application, ou en cliquant sur le logo présent en haut à gauche de la page, dans la barre de menu. (cf. Figure 1).

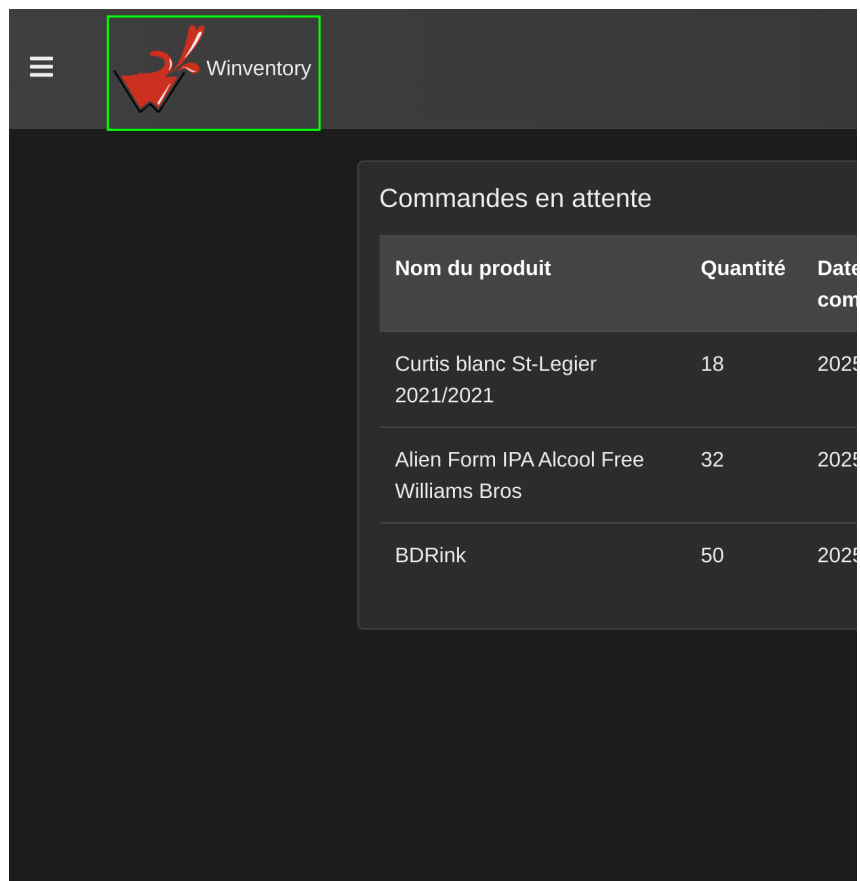


Figure 1: Utilisateur enregistré

Les autres pages sont accessibles depuis le menu déroulant présent dans la barre de menu (cf. Figures 2 et 3)

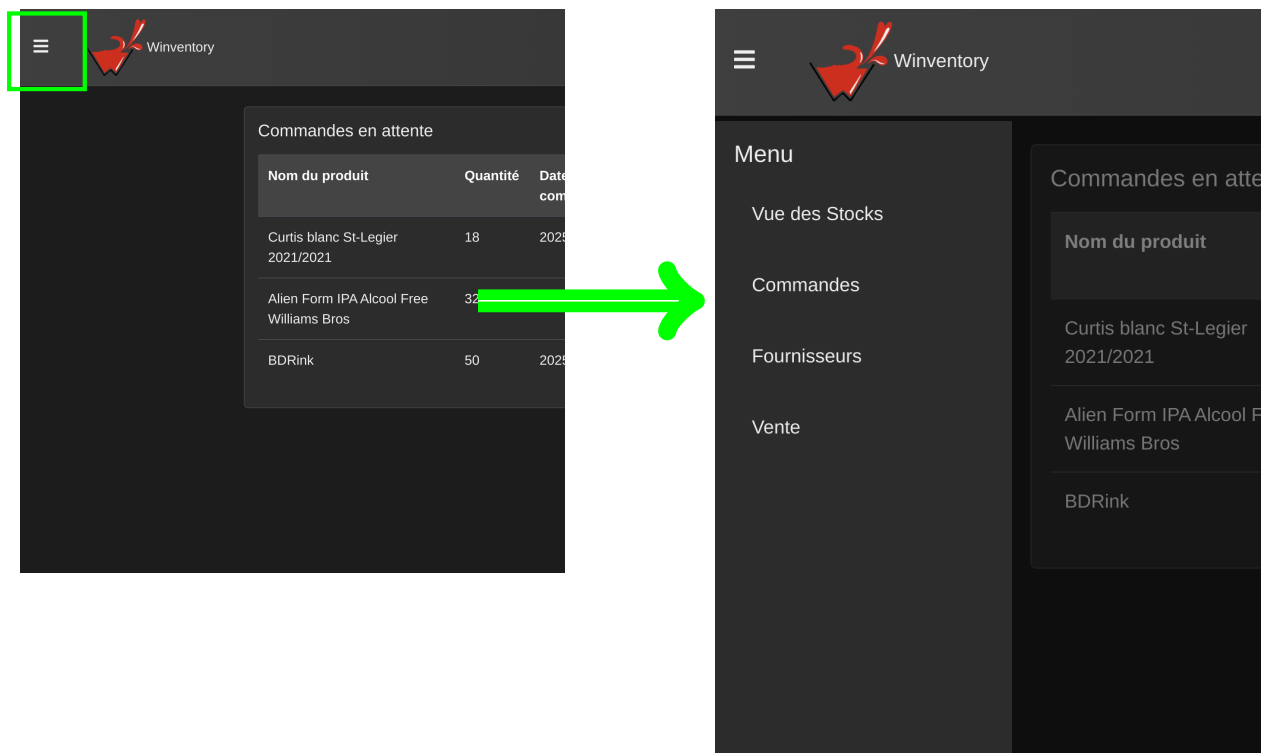


Figure 2: Menu déroulant fermé
Figure 2.1: Menu déroulant ouvert

Page de connexion

La page d'accueil comporte deux champs de texte et deux boutons:



The image shows a login page for 'Winventory'. It has a dark grey background with white text. At the top, it says 'Bienvenue sur Winventory'. Below that, it says 'Pour accéder au site, merci de vous connecter :'. There are two input fields: one for 'Nom d'utilisateur' and one for 'Mot de passe'. Below the password field, there is a small line of text: 'Nous ne partagerons peut-être jamais vos données avec qui que ce soit. Sauf s'il paie bien.' At the bottom, there are two buttons: 'Créer un compte' (grey) and 'Connexion' (blue). At the very bottom, there is a copyright notice: '© 2025 Winventory. Tous droits réservés.'

Figure 3: Page de connexion

Si l'utilisateur a déjà un profil, il peut directement se connecter en remplissant les champs et en appuyant sur le bouton "Connexion". Si ce n'est pas le cas, l'utilisateur peut se créer un compte en cliquant sur le bouton "Créer un compte":

Lorsque l'utilisateur clique sur le bouton "Créer un compte", il est redirigé sur la page d'enregistrement de l'utilisateur.

Page d'enregistrement

Cette page permet à l'utilisateur de s'enregistrer pour avoir accès à l'application.



Créer un compte

Inscrivez-vous pour accéder à Winventory

Nom d'utilisateur

Mot de passe

Retour S'inscrire

© 2025 Winventory. Tous droits réservés.

Figure 4: Page d'enregistrement

Pour s'enregistrer, il suffit simplement de trouver un nom et un mot de passe et de cliquer sur le bouton "S'inscrire". Lorsque le nouvel utilisateur clique sur le bouton "S'inscrire", un label rouge s'affiche, indiquant si les informations de l'utilisateur sont déjà utilisées ou si l'utilisateur a été correctement enregistré.



Créer un compte

Inscrivez-vous pour accéder à Winventory

Nom d'utilisateur

magicWorld

Mot de passe

Retour S'inscrire

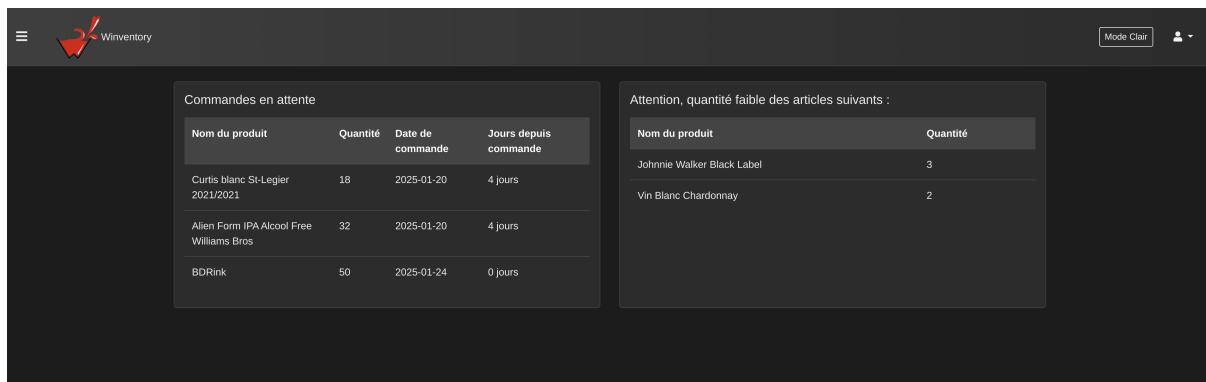
User registered successfully

© 2025 Winventory. Tous droits réservés.

Figure 5: Utilisateur enregistré

L'utilisateur peut alors retourner sur la page de connexion et se connecter avec le login qu'il vient de créer.

Page d'accueil



The screenshot shows the Winventory dashboard. At the top left is a menu icon and the Winventory logo. At the top right are a 'Mode Clair' button and a user profile icon. The main content area is divided into two panels. The left panel, titled 'Commandes en attente', contains a table with pending orders. The right panel, titled 'Attention, quantité faible des articles suivants :', contains a table of items with low stock levels.

Nom du produit	Quantité	Date de commande	Jours depuis commande
Curtis blanc St-Legier 2021/2021	18	2025-01-20	4 jours
Allen Form IPA Alcohol Free Williams Bros	32	2025-01-20	4 jours
BDRink	50	2025-01-24	0 jours

Nom du produit	Quantité
Johnnie Walker Black Label	3
Vin Blanc Chardonnay	2

Figure 6: Page d'accueil

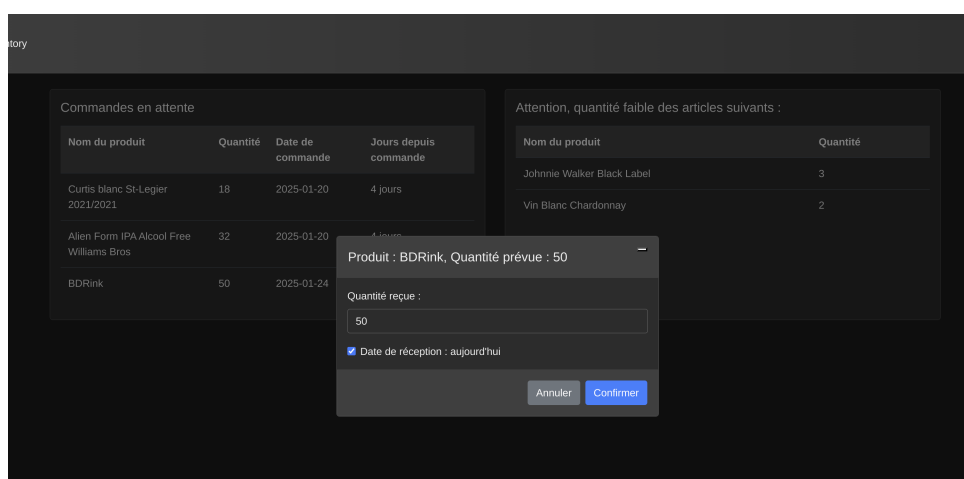
Sur cette page, on peut voir deux tableaux:

1. Les commandes en attente
2. Les articles dont la quantité en stock est basse

Commandes en attente

Ce tableau permet de percevoir toutes les commandes qui ont été passées mais qui ne sont pas encore "arrivées", elles sont donc en attente et il faut les valider pour que leurs valeurs soient enregistrées et appliquées dans la vue des stocks.

Pour les approuver, il suffit de cliquer sur l'article dont il est question et la fenêtre suivante s'affiche:



The screenshot shows the 'Approuver la commande' dialog box. It has a title bar 'Produit : BDRink, Quantité prévue : 50'. Below the title bar is a text input field for 'Quantité reçue :'. The input field contains the value '50'. Below the input field is a checkbox labeled 'Date de réception : aujourd'hui', which is checked. At the bottom of the dialog box are two buttons: 'Annuler' and 'Confirmer'.

Figure 7: Approuver la commande

On peut alors modifier la quantité reçue si cette dernière ne correspond pas à la livraison réelle et modifier la date de réception (si jamais la modification des stocks est faite en retard).

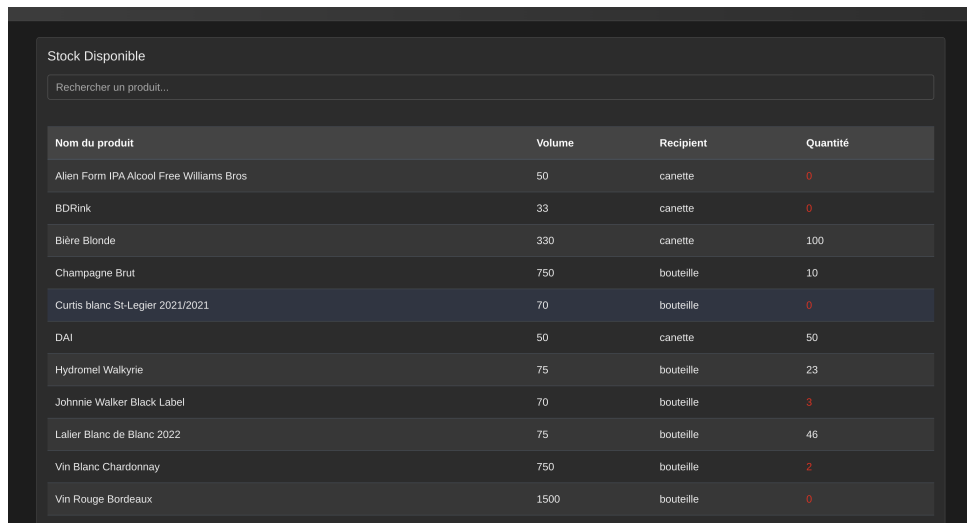
Lorsque la commande est enregistrée, elle disparaît du tableau "Commandes en attente".

Articles dont les stocks sont bas

Ce tableau affiche simplement les articles dont la quantité est faible. Cela permet de passer commande auprès des fournisseurs avant d'être en rupture de stock.

Vue des stocks

Cette page affiche les différents articles en stock, on retrouve leur nom, leur volume, le type de récipient (canette/bouteille) et la quantité qu'il y a en stock.



Nom du produit	Volume	Recipient	Quantité
Alien Form IPA Alcohol Free Williams Bros	50	canette	0
BDRink	33	canette	0
Bière Blonde	330	canette	100
Champagne Brut	750	bouteille	10
Curtis blanc St-Legier 2021/2021	70	bouteille	0
DAI	50	canette	50
Hydromel Walkyrie	75	bouteille	23
Johnnie Walker Black Label	70	bouteille	3
Lalier Blanc de Blanc 2022	75	bouteille	46
Vin Blanc Chardonnay	750	bouteille	2
Vin Rouge Bordeaux	1500	bouteille	0

Figure 8: Vue des stocks

Lorsqu'une commande est en attente, elle apparaît également dans la vue des stocks, avec une quantité valant 0. La quantité est ensuite mise à jour lorsque la commande est approuvée.

Il est également possible de rechercher des produits via la barre de recherche. Il est donc possible de filtrer par nom, volume, type de récipient ou quantité.

Formulaire de commandes

Cette page sert à enregistrer une commande qui a été passée auprès d'un ou plusieurs fournisseurs. Il est possible d'inclure plusieurs produits/articles en rajoutant des champs via le bouton "Ajouter un produit". Il est également possible d'enlever un champ avec le bouton "Retirer un produit" si un champ de trop a été ajouté. Il doit y avoir au minimum un champ pour effectuer la commande.

The image displays two versions of a 'Commande' (Order) form. The left version shows a single product entry labeled 'Produit N°1'. It includes a date selector at the top set to '01/25/2025' with a checkbox for 'Date du jour'. The product entry form contains fields for 'Produit', 'Volume (cl)', 'Récipient', 'Fournisseur', 'Date Fin de Vente', 'Date Pénemption', 'Taux d'alcool (%)', 'Quantité', and 'Prix'. At the bottom are buttons for 'Retirer un produit', 'Ajouter un produit', and 'Soumettre'. The right version shows the same form but with two product entries, 'Produit N°1' and 'Produit N°2', stacked vertically, demonstrating the 'Ajouter un produit' functionality.

Figure 9: Formulaire d'enregistrement de commande à un champ

Figure 9.1: Formulaire d'enregistrement de commande à plusieurs champs

Il faut obligatoirement remplir tous les champs pour pouvoir enregistrer la commande. Si la commande a été faite le jour même, il suffit de cocher la case pour que le champ soit rempli automatiquement, il est alors impossible de changer manuellement la date. Cette option est mise en place par défaut.

Lorsqu'on ajoute un nouvel article, ce dernier est ajouté dans la base de données en tant que produit et article.

Ce formulaire sert donc à ajouter des articles dans la base de données (mouvements de stock entrant).

Formulaire de ventes

Ce formulaire permet de simuler une vente faite par un vendeur auprès d'un client. Il faut donc renseigner la date de la vente, le nom du vendeur (un vendeur existant), l'email du client (existant déjà également), ainsi que tous les autres champs décrivant l'article.

Tout comme le formulaire de commande, il est possible d'ajouter et de retirer des champs pour les différents articles. Chaque vente est effectuée par un seul vendeur et auprès d'un seul client.

The image displays two versions of a 'Vente' (Sales) form. Both forms have a title 'Vente' in blue. They include input fields for 'Date de la Vente' (with a date picker showing 01/25/2025 and a checked 'Date du jour' checkbox), 'Nom du Vendeur' (with a 'Nom' input), and 'Email du client' (with an 'email' input). The left form features a single product section 'Produit N°1' with fields for 'Produit' (a dropdown menu), 'Volume (cl)' (value 33), 'Taux d'alcool (‰)' (value 5.0), 'Quantité' (value 0), 'Prix' (value 0.3), and 'Récipient' (a dropdown menu showing 'Canette'). The right form has two identical sections, 'Produit N°1' and 'Produit N°2'. Both forms show a 'Total de la vente' of 0.00 at the bottom, along with buttons for 'Retirer un produit' (red), 'Ajouter un produit' (blue), and 'Effectuer la Vente' (green).

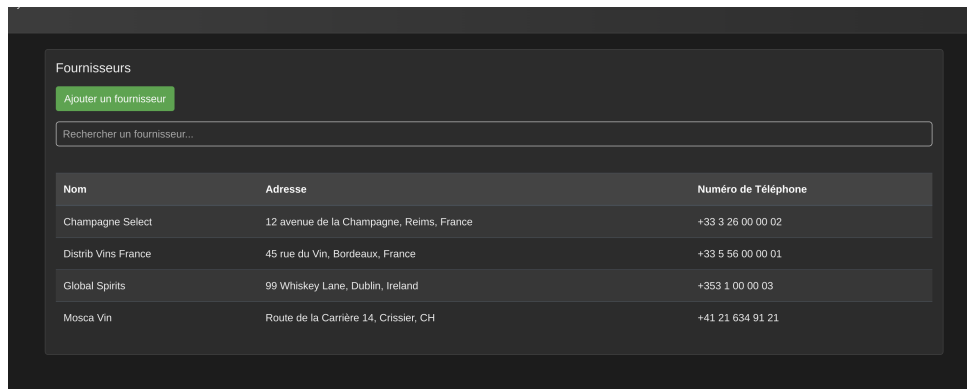
Figure 10: Formulaire d'enregistrement de vente à un champ

Figure 10.1: Formulaire d'enregistrement de vente à plusieurs champs

Le prix total, en fonction de la quantité et du prix de chaque produit, est affiché en bas de la page.

Fournisseurs

Tous les fournisseurs du magasin sont affichés sur cette page.



The screenshot shows a web interface for managing suppliers. At the top, there is a green button labeled 'Ajouter un fournisseur'. Below it is a search bar with the placeholder text 'Rechercher un fournisseur...'. A table lists five suppliers with their names, addresses, and phone numbers.

Nom	Adresse	Numéro de Téléphone
Champagne Select	12 avenue de la Champagne, Reims, France	+33 3 26 00 00 02
Distrib Vins France	45 rue du Vin, Bordeaux, France	+33 5 56 00 00 01
Global Spirits	99 Whiskey Lane, Dublin, Ireland	+353 1 00 00 03
Mosca Vin	Route de la Carrière 14, Crissier, CH	+41 21 634 91 21

Figure 11: Page fournisseur

Si le fournisseur n'existe pas encore dans la base de données, il est possible d'accéder au formulaire d'ajout des fournisseurs via le bouton "Ajouter un fournisseur".

Il est également possible de rechercher un fournisseur via la barre de recherche.

Dans le cas où les informations fournies pour le fournisseur sont incorrectes, il est possible de supprimer le dit-fournisseur et de le re-cr  er par la suite:

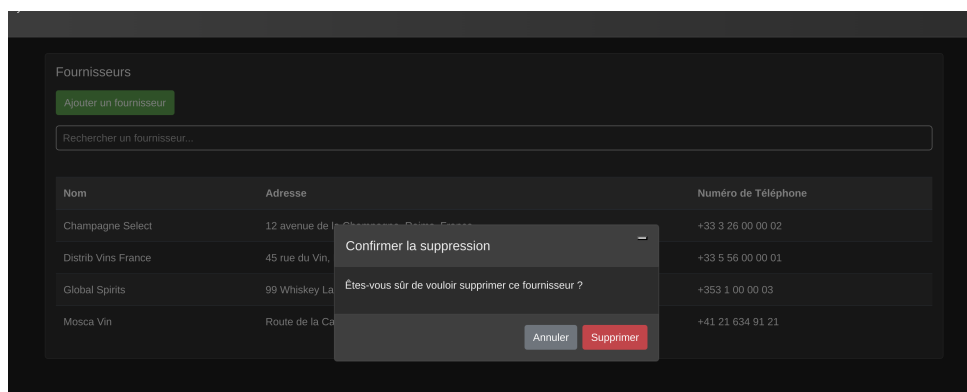


Figure 12: Supprimer un fournisseur

Ajouter un fournisseur

Cette page est accessible depuis la page fournisseur, via le bouton "Ajouter un fournisseur".

Il est alors simplement nécessaire de remplir les champs demandant le nom, l'adresse et le numéro de téléphone du fournisseur.

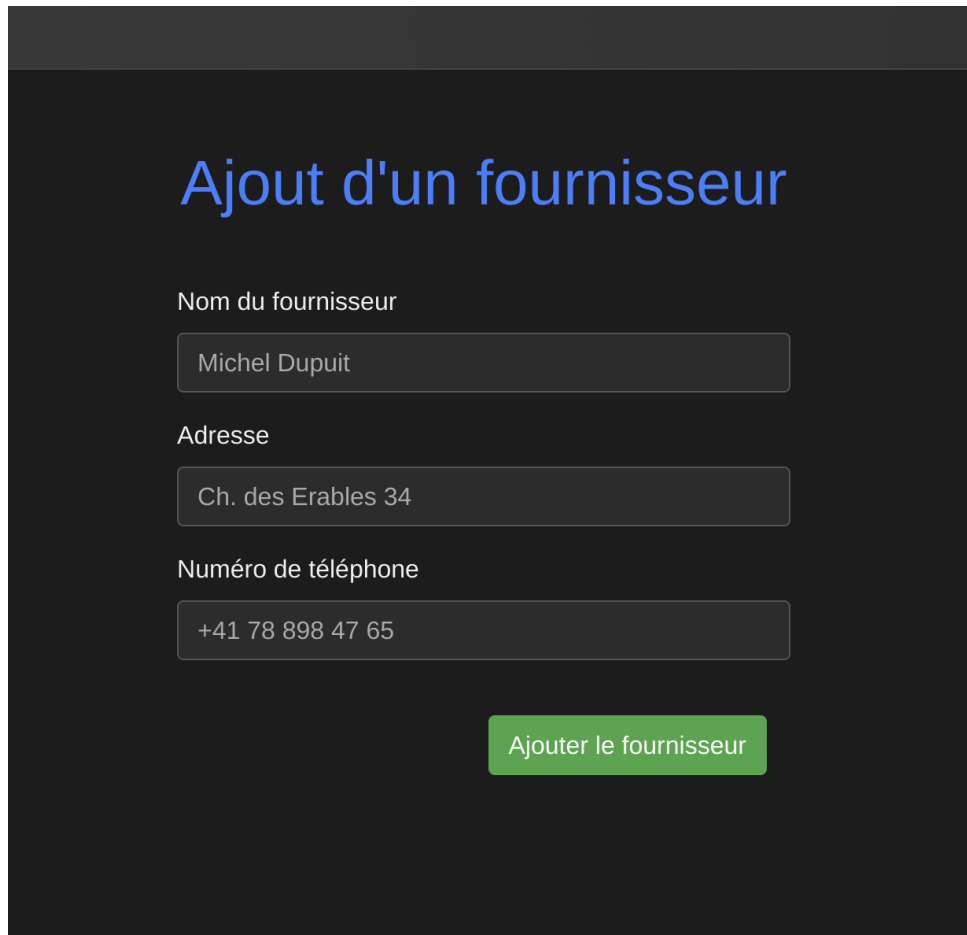
The image shows a web form titled "Ajout d'un fournisseur" in a blue font. The form is set against a dark background. It contains three input fields, each with a label above it: "Nom du fournisseur" with the value "Michel Dupuit", "Adresse" with the value "Ch. des Erables 34", and "Numéro de téléphone" with the value "+41 78 898 47 65". All input fields have a light gray border. At the bottom right of the form is a green button with the text "Ajouter le fournisseur" in white.

Figure 13: Page fournisseur

Spécificité

Si vous êtes partis pris de la lumière, notre affichage "dark mode" a dû vous piquer les yeux.

Rien que pour vous, nous avons implémenter un mode clair (light mode). Pour l'afficher, il faut simplement cliquer sur le bouton "Mode clair" dans la barre de menu:

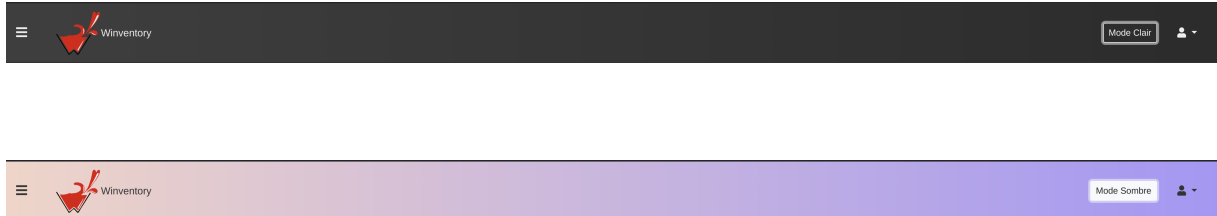


Figure 14: Dark mode
Figure 14.1: Light mode

Bugs connus

- Une vente peut être effectuée:
 - par un vendeur n'étant pas enregistré dans la base de données
 - avec l'email quelconque d'un potentiel client
 - pour un produit n'existant pas dans les stocks.
- N'importe qui peut se connecter à l'application à condition de s'enregistrer.
- Il est impossible de référencer la provenance d'un produit (manquement dans l'implémentation du frontend)

Perspectives d'avenir

L'application telle qu'elle existe aujourd'hui n'apporte pas toutes les spécificités que nous aurions voulu implémenter. Nous continuerons le développement de l'application **Winventory** avec les points suivants:

- Ajouter la page "profil" pour les vendeurs, page référençant leurs informations et étant liée au login (seuls les vendeurs actifs ou managers, à décider, auront accès à l'application).
- Ajouter une page référençant tous les clients et une page permettant de les enregistrer dans la base de données.
- Implémenter la fonctionnalité de gérer plusieurs magasins au lieu d'un seul.
- Ajouter une couche de sécurité afin que l'application soit "sûre".

- Ajouter une page permettant d’afficher les statistiques des différents produits, lesquels sont les plus ou moins vendus
- Implémenter les points fidélité pour les clients
- Ajouter un attribut aux produits afin de pouvoir par la suite les modifier en fonction de potentiels rabais.

Conclusion

Annexes

.1 Annexe 1 - Accès à l'application, DAI

(page suivante)

Winventory - Gestion Intelligente des Stocks de Boissons

Table des matières

-  [Équipe](#)
-  [Description du Projet](#)
-  [Fonctionnalités Principales](#)
-  [Objectif](#)
-  [Compilation, édition et lancement du projet](#)
-  [Documentation de l'API](#)
-  [Utilisation de l'application web avec cURL](#)
-  [Installation et Configuration d'une Machine Virtuelle](#)
-  [Configuration de la zone DNS pour accéder à l'application web avec DuckDNS](#)
-  [Détails de l'API](#)

Équipe

- **Lestiboudois Maxime**
- **Parisod Nathan**
- **Surbeck Léon**

Description du Projet

Winventory est une application web conçue pour faciliter la gestion des stocks de boissons, en particulier pour les cavistes, bars, restaurants et distributeurs. Grâce à une interface intuitive, Winventory permet de suivre les entrées et sorties de stock, gérer les fournisseurs, et optimiser l'approvisionnement en fonction des ventes et des besoins.

Fonctionnalités Principales

Gestion des Stocks

- Visualisation en temps réel des articles en stock.
- Alertes automatiques pour les produits à faible quantité.

Gestion des Commandes et Approvisionnements

- Consultation des commandes en attente.
- Mise à jour des réceptions de marchandises avec validation des quantités.

Gestion des Fournisseurs

- Ajout des fournisseurs via une interface dédiée.

Interface Web Moderne et Ergonomique

- Navigation fluide et design optimisé pour une expérience utilisateur agréable.
- Thème clair/sombre pour un confort visuel personnalisé.
- Menu interactif avec accès rapide aux différentes sections.

Technologie

- API RESTful basée sur **Javalin** et **PostgreSQL**.
- Déploiement avec **Docker & Traefik** pour une infrastructure robuste et scalable.

Objectif

Winventory vise à **simplifier** la gestion des stocks de boissons en offrant un suivi précis et une visibilité accrue sur les stocks, les commandes et les fournisseurs.

Que vous soyez un **gérant de bar**, un **responsable de stock**, ou un **caviste**, **Winventory** vous aide à éviter les ruptures de stock et à optimiser vos commandes pour une gestion plus efficace.

Gérez vos stocks intelligemment avec Winventory !

Compilation, édition et lancement du projet

Note : Il vous faudra Java temurin 21 et Maven installés sur votre machine pour compiler et lancer ce projet.

Vous pouvez les installer facilement grâce à [ce guide](#)

Compiler et lancer le projet en direct sur la machine hôte

Note : pour run le projet en local, il faut avoir une base de données PostgreSQL avec les réglages suivants (N'oubliez

pas de mettre à jour le mot de passe dans le fichier `src/main/java/ch/heigvd/dai/Main.java`):

- **Nom de la Base de données** : `winventory`
- **Nom d'utilisateur** : `postgres`
- **Mot de passe** : `WhateverYouWant`
- **Port** : `5432`

Le script SQL pour créer et remplir la base de données avec des données de base se trouve dans le dossier `db-scripts`.

Vous devez les exécuter dans l'ordre pour vous assurer que l'application fonctionne correctement et que les données sont correctement insérées.

Une fois cela fait, vous pouvez suivre les étapes suivantes pour compiler et lancer le projet en local :

1. **Cloner le dépôt** : `git clone git@github.com:La-Kirby-Team/BDR-Project.git`
2. **(Optionnel) Éditions** : Faites les modifications nécessaires dans le code source.
3. **Compiler le projet** : `mvn clean package`
4. **Lancer l'application** : `java -jar target/Winventory-0.9.jar`



Lancer le Projet avec Docker compose (DB incluse)

Note : pour run le projet avec Docker, il faut avoir [Docker](#) et [Docker-compose](#) installés sur votre machine.

1. **Cloner le dépôt** : `git clone git@github.com:La-Kirby-Team/BDR-Project.git`
2. **(Optionnel) Éditions** : Faites les modifications nécessaires dans le code source.
3. **Compiler le projet** : `mvn clean package`
4. **Construire l'image Docker** : `docker build -t winventory .`
5. **Lancer les conteneurs Docker (Vérifiez bien que votre terminal se trouve dans le dossier racine du projet)** :
`docker compose up`
6. **(Optionnel) Démarrer uniquement le serveur web** :
`docker run --rm --name winventory_web -p 80:8080 winventory:latest` (Assurez-vous qu'une base de données PostgreSQL soit accessible depuis le conteneur Docker)

Une fois cela fait, votre serveur web devrait être accessible à l'adresse `http://localhost` si vous avez lancé le serveur web en local, sinon, vous pouvez accéder à l'application web via l'adresse que vous devez changer dans le `docker-compose.yml` (les règles de redirections de Traefik).



Publier l'Application avec Docker

Pour publier une image Docker sur ghcr.io, il vous faudra un compte GitHub et un token d'authentification [cliquez ici](#) pour la marche à suivre en anglais pour en créer / récupérer un (Chapitre Authenticating with a personal access token (classic)).

1. **Cloner le dépôt** : `git clone git@github.com:La-Kirby-Team/BDR-Project.git`
2. **(Optionnel) Éditions** : Faites les modifications nécessaires dans le code source.
3. **Compiler le projet** : `mvn clean package`

4. **Construire l'image Docker** : `docker build -t wininventory .`

5. **Se connecter à GitHub Container Registry** : `docker login ghcr.io -u`
`VotrePseudoGithubICI` puis entrez votre token
d'authentification.

6. **Taguer l'image pour le dépôt Docker Hub** : `docker tag wininventory`
`ghcr.io/VotrePseudoGithubICI/wininventory:latest`

7. **Pousser l'image vers Docker Hub** : `docker push`
`ghcr.io/VotrePseudoGithubICI/wininventory`

Assurez-vous de remplacer `VotrePseudoGithubICI` par votre nom d'utilisateur GitHub.

Une fois l'image publiée, vous pouvez la déployer sur n'importe quel serveur en utilisant Docker :

1. **Tirer l'image depuis Docker Hub** : `docker pull`
`ghcr.io/VotrePseudoGithubICI/wininventory:latest`

2. **Lancer un conteneur avec l'image** : `docker run --rm --name wininventory_web -p`
`80:8080 wininventory:latest`

Assurez vous qu'une base de données PostgreSQL soit accessible depuis le conteneur Docker, le `docker-compose.yml` qui est
fourni vous permettra de lancer le projet ainsi que sa base de données.
Pour faire fonctionner le docker-compose, il suffit de lancer la commande `docker-compose up`
dans le dossier racine du
projet.

Cela permettra de déployer l'application et sa base de données sur votre serveur.

Documentation de l'API

L'API RESTful de Winventory permet d'interagir avec les différentes fonctionnalités de l'application.
Voici un aperçu
des principales routes disponibles :

Authentification

- `POST /api/login` : Authentification de l'utilisateur.
- `POST /api/logout` : Déconnexion de l'utilisateur.
- `PUT /api/register` : Inscription d'un nouvel utilisateur.

Vue des Stocks

- `GET /api/stock` : Récupérer la liste des articles en stock.

Gestion des Commandes

- `GET /api/orders-waiting` : Récupérer la liste des commandes en attente (pas encore livrées).
- `PUT /api/orders-confirm` : Confirmation de la livraison d'une commande.

Gestion des Fournisseurs

- `GET /api/providers` : Récupérer la liste des fournisseurs.
- `POST /api/providers` : Ajouter un nouveau fournisseur.
- `DELETE /api/providers/{id}` : Supprimer le fournisseur avec l'ID spécifié.

Toutes les routes nécessitent une authentification préalable via le endpoint `/api/auth/login`.

Pour plus de détails,
cliquez [ici](#)

Utilisation de l'application web avec cURL

Voici quelques exemples de commandes cURL pour interagir avec l'API RESTful de Winventory.

Authentification

Requête :

```
curl -X POST http://localhost/api/login -H "Content-Type: application/json" -d '{"username":"utilisateur1","password":"motdepasse123"}'
```

Réponse :

```
{
  "message": "Logged in successfully"
}
```

Requête :

```
curl -X POST http://localhost/api/logout -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Réponse :

```
{
  "message": "Logged out successfully"
}
```

Requête :

```
curl -X PUT http://localhost/api/register -H "Content-Type: application/json" -d '{"username":"nouvelUtilisateur","password":"motdepasse456","email":"email@example.com"}'
```

Réponse :

```
{
  "message": "User registered successfully"
}
```



Vue des Stocks

Requête :

```
curl -X GET http://localhost/api/stock
```

Réponse :

```
[
  [
    "1",
    "Vin Rouge",
    "750",
    "Bouteille",
    "20"
  ],
  [
    "2",
    "Vin Blanc",
    "500",
    "Bouteille",
    "5"
  ],
  [
    "3",
    "Champagne",
    "750",
    "Bouteille",
    "2"
  ]
]
```



Gestion des Commandes

Requête :

```
curl -X GET http://localhost/api/orders-waiting
```

Réponse :

```
[
  {
    "produit": "Vin Rouge",
    "quantite": 50,
    "dateCommande": "2023-10-01",
    "joursDepuisCommande": 10,
    "mouvementStockId": 1
  },
  {
    "produit": "Vin Blanc",
    "quantite": 30,
    "dateCommande": "2023-10-05",
    "joursDepuisCommande": 6,
    "mouvementStockId": 30
  }
]
```

```
"mouvementStockId": 2
}
]
```

Requête :

```
curl -X PUT http://localhost/api/orders-confirm -H "Content-Type: application/json" -d '{"id":1,"date":"2023-10-11","quantite":50}'
```

Réponse :

```
{
  "message": "Commande confirmée avec succès."
}
```

Gestion des Fournisseurs

Requête :

```
curl -X GET http://localhost/api/providers
```

Réponse :

```
[
  {
    "id": 1,
    "nom": "Fournisseur A",
    "adresse": "123 Rue Principale",
    "numeroTelephone": "+41 78 123 45 67"
  },
  {
    "id": 2,
    "nom": "Fournisseur B",
    "adresse": "456 Avenue Secondaire",
    "numeroTelephone": "+41 78 765 43 21"
  }
]
```

Requête :

```
curl -X POST http://localhost/api/providers -H "Content-Type: application/json" -d '{"name":"Fournisseur C","address":"789 Boulevard Tertiaire","phone":"+41 78 987 65 43"}'
```

Réponse :

```
{
  "message": "Fournisseur ajouté avec succès."
}
```

Requête :


```
curl -X DELETE http://localhost/api/providers/1"
```

Réponse :

```
{  
  "message": "Fournisseur supprimé avec succès."  
}
```

Installation et Configuration d'une Machine Virtuelle

Ce guide vous aidera à installer et configurer Winventory, mais nous partons du principe que vous possédez déjà une machine virtuelle avec une distribution Linux installée. Si vous n'avez pas encore de machine virtuelle, vous pouvez suivre ce [guide](#) pour installer Ubuntu 20.04 LTS sur VirtualBox. Vous pouvez également utiliser un fournisseur de cloud comme AWS, Azure ou Google Cloud pour créer une machine virtuelle.



Installation de Docker et Docker Compose

1. Mettre à jour l'index des paquets :

```
sudo apt update
```

2. Installer les paquets permettant à apt d'utiliser un dépôt via HTTPS :

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

3. Ajouter la clé GPG officielle de Docker :

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

4. Configurer le dépôt stable :

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

5. Mettre à jour l'index des paquets :

```
sudo apt update
```

6. Installer Docker :

```
sudo apt install docker-ce docker-ce-cli containerd.io
```

7. Vérifier que Docker est correctement installé :

```
sudo docker --version
```

8. Installer Docker Compose :

```
sudo apt install docker-compose
```

9. Vérifier que Docker Compose est correctement installé :

```
sudo docker-compose --version
```



Installation de SDKMAN!, Java SDK et Maven

SDKMAN! est un outil pratique pour gérer plusieurs versions de SDK sur votre machine. Voici comment l'installer :

1. Installer SDKMAN! :

```
curl -s "https://get.sdkman.io" | bash
```

2. **Suivre les instructions à l'écran** pour terminer l'installation. Vous devrez ouvrir un nouveau terminal ou exécuter la commande suivante pour initialiser SDKMAN! :

```
source "$HOME/.sdkman/bin/sdkman-init.sh"
```

3. Vérifier l'installation :

```
sdk version
```

4. Installer Java temurin 21 :

```
sdk install java 21.0.4-tem
```

5. Installer Maven :

```
sdk install maven
```

SDKMAN! est maintenant installé et configuré sur votre machine. Vous pouvez utiliser SDKMAN! pour gérer facilement les versions de Java et d'autres SDK nécessaires pour votre projet.

Configuration de la zone DNS pour accéder à l'application web avec DuckDNS

Pour configurer la zone DNS et accéder à votre application web via un domaine DuckDNS au lieu de son adresse ip publique, suivez les étapes ci-dessous :

1. Créer un compte DuckDNS

Rendez-vous sur [DuckDNS](#) et connectez-vous avec votre compte GitHub.

2. Ajouter un domaine

Cliquez sur le bouton `Add Domain` et choisissez un nom de domaine. Ce domaine sera utilisé pour accéder à votre application web.

3. Configurer les enregistrements DNS

Ajoutez les enregistrements DNS nécessaires pour pointer vers l'adresse IP de votre machine virtuelle.

Ajouter un enregistrement `A`

- **Nom de domaine :** `votre-domaine.duckdns.org`
- **Adresse IP :** `IP_de_votre_machine_virtuelle`

Ajouter un enregistrement `A` générique (wildcard)

- **Nom de domaine :** `*.votre-domaine.duckdns.org`
- **Adresse IP :** `IP_de_votre_machine_virtuelle`

4. Tester la résolution DNS

Depuis votre machine locale et votre machine virtuelle, testez la résolution DNS avec la commande suivante :

```
nslookup votre-domaine.duckdns.org
```

Vous devriez obtenir une réponse avec l'adresse IP de votre machine virtuelle.

5. Accéder à l'application web

Utilisez le domaine configuré pour accéder à votre application web. Par exemple, `http://votre-domaine.duckdns.org`.

En suivant ces étapes, vous pourrez configurer la zone DNS pour accéder à votre application web via DuckDNS.



Détails de l'API



Authentification

Endpoints

POST /api/login

- **Description:** Cet endpoint permet à un utilisateur de se connecter en fournissant ses identifiants.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **username:** Le nom d'utilisateur de l'utilisateur.
 - **password:** Le mot de passe de l'utilisateur.

```
{
  "username": "utilisateur1",
  "password": "motdepasse123"
}
```

- **Réponse:** La réponse contient un message de succès et un token JWT si les informations d'identification sont valides.

```
{
  "message": "Logged in successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

POST /api/logout

- **Description:** Cet endpoint permet à un utilisateur de se déconnecter en fournissant un token d'authentification valide.
- **Requête:** Il n'y a pas de données spécifiques requises dans la requête, seul le cookie de la session est supprimé.
- **Réponse:** La réponse indique si la déconnexion a été effectuée avec succès.

```
{
  "message": "Logged out successfully"
}
```

PUT /api/register

- **Description:** Cet endpoint permet de créer un nouvel utilisateur en fournissant les informations nécessaires.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **username:** Le nom d'utilisateur du nouvel utilisateur.
 - **password:** Le mot de passe du nouvel utilisateur.

```
{
  "username": "nouveauUtilisateur",
  "password": "motdepasse456"
}
```

- **Réponse:** La réponse contient un message indiquant si l'utilisateur a été enregistré avec succès.

```
{
  "message": "User registered successfully"
}
```

Fonctionnement

- **Contrôleur:** `UserController`
 - Utilise Javalin pour définir les routes liées à l'authentification.
 - Gère les connexions, déconnexions et inscriptions des utilisateurs.
 - Vérifie les identifiants et génère des tokens JWT pour les connexions réussies.

Exemple de Réponses JSON

Connexion réussie:

```
{
  "message": "Logged in successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Déconnexion réussie:

```
{
  "message": "Logged out successfully"
}
```

Inscription réussie:

```
{
  "message": "User registered successfully"
}
```



Vue des Stocks

Endpoints

GET /api/stock

- **Description:** Cet endpoint permet de récupérer la liste des articles en stock.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.

- **Réponse:** La réponse est une liste d'articles en stock, chaque article étant représenté par un tableau de chaînes de caractères. Chaque tableau contient les informations suivantes :
 - **ID:** L'identifiant unique de l'article.
 - **Nom du produit:** Le nom du produit.
 - **Volume:** Le volume de l'article.
 - **Recipient:** Le récipient de l'article.
 - **Quantité:** La quantité disponible de l'article.

GET /api/articles-lowQT

- **Description:** Cet endpoint permet de récupérer la liste des articles dont la quantité est faible.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse est similaire à celle de l'endpoint `/api/stock`, mais ne contient que les articles dont la quantité est considérée comme faible.

Fonctionnement

- **Contrôleur:** `StockController`
 - Le contrôleur utilise Javalin pour définir les routes de l'API.
 - Il utilise une connexion à une base de données asynchrone pour exécuter des requêtes SQL et récupérer les données.
- **Requêtes SQL:**
 - `stockQuery.sql`: Cette requête SQL récupère les informations sur les articles en stock en joignant plusieurs tables (`Produit`, `Article`, `MouvementStock`) et en utilisant des fonctions de regroupement et de coalescence pour obtenir les données nécessaires.
 - `lowQTArticles.sql`: Cette requête SQL est similaire à `stockQuery.sql`, mais elle filtre les articles pour ne récupérer que ceux dont la quantité est faible.

Exemple de Réponse JSON

```
[
  [
    "1",
    "Vin Rouge",
    "750",
    "Bouteille",
    "20"
  ],
  [
    "2",
    "Vin Blanc",
    "500",
    "Bouteille",
    "5"
  ]
]
```

```

],
[
  "3",
  "Champagne",
  "750",
  "Bouteille",
  "2"
]
]

```

Gestion des Commandes

Endpoints

GET /api/orders-waiting

- **Description:** Cet endpoint permet de récupérer la liste des commandes en attente.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse est une liste de commandes en attente, chaque commande étant représentée par un objet JSON contenant les informations suivantes :
 - **produit:** Le nom du produit commandé.
 - **quantite:** La quantité commandée.
 - **dateCommande:** La date de la commande.
 - **joursDepuisCommande:** Le nombre de jours écoulés depuis la commande.
 - **mouvementStockId:** L'identifiant unique du mouvement de stock associé à la commande.

PUT /api/orders-confirm

- **Description:** Cet endpoint permet de confirmer la réception d'une commande.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **id:** L'identifiant unique du mouvement de stock.
 - **date:** La date de réception de la commande.
 - **quantite:** La quantité reçue.
- **Réponse:** La réponse indique si la mise à jour de la commande a été effectuée avec succès ou si une erreur s'est produite.

Fonctionnement

- **Contrôleur:** `OrderController`
 - Le contrôleur utilise Javalin pour définir les routes de l'API.
 - Il utilise une connexion à une base de données asynchrone pour exécuter des requêtes SQL et récupérer les données.
- **Requêtes SQL:**

- `waitingOrders.sql`: Cette requête SQL récupère les informations sur les commandes en attente en joignant plusieurs tables (`Approvisionnement`, `MouvementStock`, `Article`, `Produit`) et en utilisant des fonctions de calcul pour obtenir les données nécessaires.

Exemple de Réponse JSON

```
[
  {
    "produit": "Vin Rouge",
    "quantite": 50,
    "dateCommande": "2023-10-01",
    "joursDepuisCommande": 10,
    "mouvementStockId": 1
  },
  {
    "produit": "Vin Blanc",
    "quantite": 30,
    "dateCommande": "2023-10-05",
    "joursDepuisCommande": 6,
    "mouvementStockId": 2
  }
]
```

Gestion des Fournisseurs

Endpoints

GET /api/providers

- **Description:** Cet endpoint permet de récupérer la liste des fournisseurs.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse est une liste de fournisseurs, chaque fournisseur étant représenté par un objet JSON contenant les informations suivantes :
 - **id:** L'identifiant unique du fournisseur.
 - **nom:** Le nom du fournisseur.
 - **adresse:** L'adresse du fournisseur.
 - **numeroTelephone:** Le numéro de téléphone du fournisseur.

POST /api/providers

- **Description:** Cet endpoint permet d'ajouter un nouveau fournisseur.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **name:** Le nom du fournisseur.
 - **address:** L'adresse du fournisseur.
 - **phone:** Le numéro de téléphone du fournisseur.
- **Réponse:** La réponse indique si l'ajout du fournisseur a été effectué avec succès ou si une erreur s'est produite.

DELETE /api/providers/{id}

- **Description:** Cet endpoint permet de supprimer un fournisseur avec l'ID spécifié.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse indique si la suppression du fournisseur a été effectuée avec succès ou si une erreur s'est produite.

Fonctionnement

- **Contrôleur:** `ProviderController`
 - Le contrôleur utilise Javalin pour définir les routes de l'API.
 - Il utilise une connexion à une base de données asynchrone pour exécuter des requêtes SQL et récupérer les données.
- **Requêtes SQL:**
 - `providerQuery.sql`: Cette requête SQL récupère les informations sur les fournisseurs en les triant par nom.
 - `providerNew.sql`: Cette requête SQL insère un nouveau fournisseur dans la base de données.
 - `deleteProviderQuery.sql`: Cette requête SQL supprime un fournisseur de la base de données en fonction de son ID.

Exemple de Réponse JSON

```
[
  {
    "id": 1,
    "nom": "Fournisseur A",
    "adresse": "123 Rue Principale",
    "numeroTelephone": "+41 78 123 45 67"
  },
  {
    "id": 2,
    "nom": "Fournisseur B",
    "adresse": "456 Avenue Secondaire",
    "numeroTelephone": "+41 78 765 43 21"
  }
]
```